Durham University

CIUK Student Cluster Competition Challenge
Durham University Subchallenge

Alastair Basden Karen Bower Chung Ming Loi Mark R. Lovell
Wanqing Tu Tobias Weinzierl

October 14, 2026

1 Introduction

The purpose of this CIUK student cluster competition challenge is to encourage students to explore the interplay between bandwidth and compute bounds. By combining theory with hands-on activities, students will showcase a firm understanding of how the bandwidth and compute capabilities of a compute node in a cluster perform, as compared to the theoretical specifications published by vendors. This will motivate students to improve the source code, i.e., the first steps towards performance engineering, although any change of source code is out-of-scope.

2 The machines

We will use nodes from the DINE and DINE2 clusters in the HPC Hardware and Development Lab @Durham, which are part of the COSMA HPC facility hosted in Durham.

Access to the machines will be remote. In order to access the hardware lab you will each need to follow the online instructions to register for DiRAC SAFE and then request a machine account on COSMA using project code do022. You may request an account at any time, including before the start of the challenge on Monday 19th October, but note that the staff who support COSMA and create accounts work regular office hours only.

Once your account has been created, login to Cosma8 (login8.cosma.dur.ac.uk and submit jobs to DINE/DINE2 from there using the Slurm queues `ciuk-dine` and ciuk-dine2. You will need to specify "#SBATCH -A do022" and "#SBATCH -p ciuk-dine" (or ciuk-dine2) in your jobs.

The job submission queues will open on Monday 19th October and close at 4pm on Friday 23rd October. Jobs that are submitted within that time but have not completed will still run. The maximum duration for a job is 1 hour.

We recommend groups to rely on an external service (e.g. github) to archive, share and version control their data. If you create shared directories on COSMA, use the setfacl command to control access so that other groups cannot see your data.

Please note that all submitted material should list your team name and all user names associated with it.

3 Task 1: Likwid

Familiarise yourself with LIKWID and its application `likwid-bench`. Find out what the maximum compute capability of each node and its maximal bandwidth are. Measure the throughput and MFLOPS/s you obtain with the benchmark at each node and compare the results to develop your insights.

4 Task 2: Grid—a Lattice QCD code

Grid is a data parallel C++ QCD library, for simulating quantum chromodynamics (the theory of quarks and gluons) on a discrete space-time grid using supercomputers. Grid uses a four-dimensional Cartesian grid to compute properties like hadron masses and Standard Model inputs from first principles. Some rationale of grid can be found in its release paper.

For the present challenge, we do not use Grid's MPI or GPU facilities. We focus on the CPU variant. This implies that we use $1 \times 1 \times 1 \times 1$ domains. For a multi-GPU or multi-CPU version, you would patch together multiple of these domains in space-time. Here, we only stick to one setup. This can be achieved by passing `--mpi=1,1,1,1` to the code. The size of the individual patch is set via `--max-L 48` (picking a 48^4 patch; 32 and 24 are other popular choices).

Grid is part of the UK's living benchmarks (see https://github.com/ukri-bench/benchmark-grid, from where you can also find a Spack installation script), and we use setups as published for the procurement of the UK NNSS service: https://github.com/UKNNSS-Benchmarks/uknnss-benchmark-grid. You are supposed to download and install the code and then run

```
Benchmark_Grid --no-benchmark-flops-fp64 --no-benchmark-flops-sp4-2as \\
  --no-benchmark-flops-sp4-f --no-benchmark-flops-su4 --mpi=1,1,1,1 --max-L 48
```

The benchmark yields the GFlops per second per node (which is only 1 here in this setup).

- Run the code for different lattice sizes (at least 24 and 32) and shared memory parallelism (should be enabled by default).
- Vary the OpenMP thread count by either modifying the `--threads` argument of Grid or setting the environment variable OMP_NUM_THREADS (recommended).
- Create a strong scaling graph for each lattice size.
- Run `likwid-perfctr` on top of (selected) Grid configurations. For on the metrics memory bandwidth and obtained MFlops/s. You can also compare these quantities to Grid's reports.

5 Timeline

- Queues will be available 19–23 October 2026.
- Questions will only be answered if sent through the official CIUK communication channel.
- Students can apply for access prior to the queue start time. Getting access can easily take 2–3 working days, so we encourage students to factor this in. If students do not request accounts ahead of time, no extension will be granted.

6 Submission

Each team is required to submit a report on their findings. This report may not be longer than two pages A4. Please summarise your observations and insights. Plots supporting key statements are encouraged. The report will be marked in combination with the presentation.

When you write up your findings, you might want to address the following questions:

- Compare Grid's compute performance to Likwid. Where and why do they differ?
- From your data, how would you classify Grid in terms within the roofline model (see resources below)?
- How well does Grid strong scale, and can this scaling be explained using insight from the previous analyses?
- Can you make statements on Grid's weak scaling?

7 Scoring

Description of findings about node capability with `likwid-bench` (node 1)	20
Description of findings about node capability with `likwid-bench` (node 2)	20
Critical evaluation of node 1 vs. node 2	10
Explanation of observed behaviour of Grid (node 1)	10
Explanation of observed behaviour of Grid (node 2)	10
Description of findings comparing the Grid to LIKWID benchmarks	10
Best-case throughput obtained with Grid	10
Time efficiency in using the testbed (e.g. number of experiments conducted on time)	10
Sum	**100**

8 Hints

- All tools/benchmarks are open source. You might want to install them somewhere locally and play around with them, before you hit the queues on Judgement Day. You can also work with your own installation on the test node if you prefer this, but you have to document what exactly you've been doing.
- Familiarise yourself with SLURM and use the specified nodes exclusively, i.e. do not share them with others.
- Be careful with the chosen software stack (compilers) and document all optimisation settings.
- You might want to look up what the Roofline model is (cmp. SHAREing training links below) and use this as a framework to argue about your results.

Links & Material

- HPC Hardware and Development Lab @ Durham: `https://durham.readthedocs.io/en/latest/hardwarelab`
- LIKWID - Like I Knew What I Am Doing: `https://rrze-hpc.github.io/likwid/Doxygen/index.html`
- STREAM: Sustainable Memory Bandwidth in High Performance Computers: `https://www.cs.virginia.edu/stream`
- SHAREing training: `https://shareing-dri.github.io/training` (notably watch the session in the HPC Foundations course on the roofline model)
- DiRAC Training Academy: `https://training-academy.dirac.ac.uk`
- COSMA documentation (for accounts)
- DINE
- DINE2
- GRID